Lottery Search System — Design Document

1. Overview

I propose PostgreSQL as the single system of record, a tiny in-memory bitmap index for wildcard matching, and lease-based reservation using `SELECT ... FOR UPDATE SKIP LOCKED` so that two users never hold the same ticket at the same time. The whole working set is small enough that the hard part is not the search itself but handing out results safely under concurrency.

Four observations drive the design:

- **There are only 1,000,000 possible numbers** (000000–999999). 10 million tickets therefore means roughly 10 tickets per number (different series, draws, or books). Pattern matching can run over the 1M number space; allocation then picks concrete tickets.
- **A pattern is a conjunction of equality predicates.** `1****5` means position 1 = 1 AND position 6 = 5. No regex or full-text engine is needed.
- **Result size is known before the query runs.** A pattern with *k* wildcards matches exactly 10^k numbers, so the system can plan the query shape in advance and short-circuit degenerate patterns such as `******`.
- **Search and allocation are one operation.** When a user searches `****23`, the product intent is "show me tickets I can buy". The results must be reserved at read time, otherwise two users will see and try to buy the same ticket.

2. Requirements and assumptions

The stated requirements are 10M+ six-digit tickets, a 6-character pattern with `*` wildcards, no duplicate simultaneous results for the same pattern, and production-grade performance. I add the following assumptions to make the design concrete.

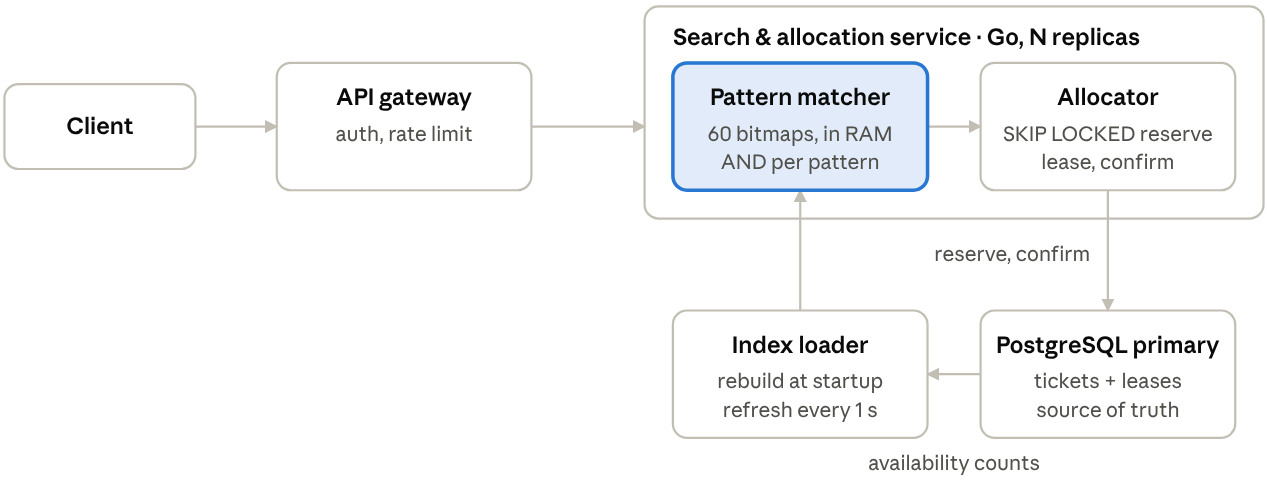
Topic	Assumption
Ticket identity	Each ticket has a unique `ticket_id`; many tickets can share the same 6-digit `number`
Ticket lifecycle	`AVAILABLE → RESERVED → SOLD`, with `RESERVED → AVAILABLE` on release or lease expiry

Topic	Assumption
Meaning of "same time"	Two users must never hold an active reservation on the same ticket. A ticket becomes visible again only after the holder buys it, releases it, or the lease expires
Result shape	A search returns a batch (default 20, max 50) of matching tickets plus a reservation token and expiry
Lease length	5 minutes by default, configurable per product rule
Write pattern	The ticket set is loaded once per draw; after that, writes are status transitions only
Scale targets	10M tickets per draw, 500 search requests/s at peak, p99 search-and-reserve latency under 50 ms
Traffic shape	Suffix (`***123`) and prefix (`12****`) patterns dominate; arbitrary patterns like `1**4*5` are rarer but must work

3. Proposed architecture

Three moving parts: a stateless Go service that holds a small in-memory index for pattern matching, PostgreSQL as the only authority on ticket state, and a loader that keeps the index close to the database.

The service matches in memory; PostgreSQL alone decides who holds a ticket



system architecture · 5 components

The matcher turns a pattern into candidate numbers without touching the database; the allocator then reserves concrete tickets inside one PostgreSQL transaction, and the loader feeds availability counts back so the matcher skips numbers that are sold out.

Request flow for `search(pattern, user_id, batch_size)`

1. Validate the pattern: exactly 6 characters, each a digit or `*`. Reject anything else.
2. Pattern matcher: AND the per-position bitmaps for the fixed positions, then AND with the availability bitmap. Result: the set of numbers that match and still have stock. Cost: microseconds.
3. Choose a random starting point inside that set (see Section 6, hot-pattern contention) and take the next few hundred candidate numbers.
4. Allocator: one PostgreSQL transaction locks and reserves up to `batch_size` available tickets among those numbers with `FOR UPDATE SKIP LOCKED`, stamping `reserved_by` and `reserved_until`.
5. Return the tickets, a reservation token, and the expiry time to the user.

`confirm(token)` moves the reserved tickets to `SOLD` with a compare-and-set on the token and expiry. `release(token)` or expiry returns them to the pool. Every endpoint accepts an idempotency key so retries cannot double-reserve or double-sell.

Redis is deliberately not in the critical path. It can be added later as a lease cache or for rate limiting, but keeping one authority for ticket state removes an entire class of consistency bugs.

4. Data model and storage choice

PostgreSQL is the recommended production store. The deciding factor is not search speed (10M rows is small for any engine) but that it offers row-level locking with `SKIP LOCKED`, transactions, and a mature operational story in one system, which is exactly what the allocation constraint needs.

Core table: `tickets`

Column	Type	Purpose
`ticket_id`	bigint, PK	Unique per physical ticket
`draw_id`	int	Which draw this ticket belongs to; also the partition key
`number`	int (0–999999)	The 6-digit value; `d1` – `d6` are derived from it, not stored
`status`	smallint	0 available, 1 reserved, 2 sold
`reserved_by`	uuid, nullable	Holder of the active lease
`reserved_until`	timestamptz, nullable	Lease expiry; NULL when not reserved
`reservation_id`	uuid, nullable	Token the client presents to confirm or release
`version`	int	Optimistic concurrency for confirm and release

A `reservations` table (`reservation_id`, `user_id`, `expires_at`, `state`, `ticket_ids[]`) records each batch handed out, so confirm and release operate on one row and the ticket rows are updated in the same transaction.

Indexes

- `(draw_id, number, ticket_id) WHERE status = 0` — a partial index covering only available tickets. This is the index the allocator walks. It shrinks as tickets sell, so lookups get faster over the life of a draw.
- `(reserved_until) WHERE status = 1` — lets the expiry sweeper find lapsed leases cheaply.
- `(reservation_id)` — confirm and release by token.

There is no per-digit index in PostgreSQL at all. Wildcard matching happens in the service (Section 5); the database only ever receives a list of concrete numbers.

Size. A row is roughly 60 bytes plus tuple overhead, so 10M tickets is about 1 GB of heap and under 500 MB of indexes. The entire working set fits in `shared_buffers` on a modest instance, which is why single-digit-millisecond allocation is realistic.

Why PostgreSQL over the alternatives

Option	Wildcard search	Atomic allocation	Operational fit	Verdict
PostgreSQL	Fine once the service supplies concrete numbers	`FOR UPDATE SKIP LOCKED`, transactions, one authority	Mature backups, replication, partitioning; widely known	Recommended
MySQL 8 / InnoDB	Same as above	`SKIP LOCKED` available since 8.0	Equivalent; choose by team familiarity	Acceptable substitute
Elasticsearch	Native wildcard queries	None; no row locks, near-real-time refresh	Would need a second system for allocation	Not for the hot path
Redis (primary)	Bitmaps via `BITOP`, Lua for atomicity	Atomic within one node, but durability and recovery are weaker for 10M primary records	Great as a cache or lease store	Optional accelerator, not the system of record
Cassandra / DynamoDB	Pattern scans awkward without many secondary indexes	Per-item conditional writes only; no multi-row transaction with skip semantics	Overkill at 10M rows	Not recommended

Partitioning `tickets` by `draw_id` keeps each active partition small and lets old draws be detached and archived without touching live data.

5. Wildcard matching: algorithm and indexing strategy

Wildcard matching is done with 60 fixed bitmaps over the 1,000,000-number space, one per (position, digit) pair. A pattern becomes a bitwise AND of the bitmaps for its fixed positions, which finishes in microseconds and never touches the database.

Index structure

- `B[p][d]` for position `p` in 1..6 and digit `d` in 0..9: a bitmap of 1,000,000 bits where bit *n* is set if the digit of number *n* at position *p* is *d*. Each bitmap is 125 KB; all 60 together are 7.5 MB. They are a pure function of arithmetic, never change, and can be built in memory in well under a second.
- `AVAIL`: one more 1,000,000-bit bitmap where bit *n* is set if at least one ticket with number *n* is still available. This is the only bitmap that changes. It is rebuilt from `SELECT number, count(*) ... WHERE status = 0 GROUP BY number` at startup and refreshed every second (or driven by `LISTEN/NOTIFY` on status changes).

Matching a pattern

1. Parse the pattern into a list of `(position, digit)` constraints, skipping `*`.
2. `result = AVAIL`; for each constraint, `result = result AND B[position][digit]`.
3. The set bits of `result` are the matching numbers that still have stock. `popcount(result)` is the number of such numbers, available for free before any allocation.

For `****23` that is two ANDs over 15,625 64-bit words each: about 30,000 word operations, or roughly 10–20 µs on one core. The result for `****23` has at most 10,000 bits set; for `123***` at most 1,000; for `******` up to 1,000,000, which is simply "any number".

From numbers to tickets. The allocator does not pass all matching numbers to the database. It picks a random bit position in `result` as a starting point, walks forward (wrapping around) and collects a window of candidate numbers, typically 100–500, then queries `tickets` with `number = ANY($candidates)` against the partial index on available rows. Because about 10 tickets share each number, a window of 100 numbers already covers a few hundred tickets, far more than one batch needs.

Why not do it all in the database?

I considered two database-only designs and would keep them as fallbacks, not the primary path:

- *Per-digit columns with six single-column B-tree indexes.* PostgreSQL can bitmap-AND these, but each index has only ten distinct values, so every index scan starts from 1M rows. A two-constraint pattern still materialises 100K row pointers per query, which is roughly 100× the work of the in-memory approach.
- *Enumerate matching numbers in SQL.* A pattern with *k* wildcards matches 10^k numbers. For k ⩽ 3 an `IN` list of up to 1,000 numbers against the partial index is cheap. For k ⩾ 4 the list grows to 10,000+ and query planning itself becomes the cost.

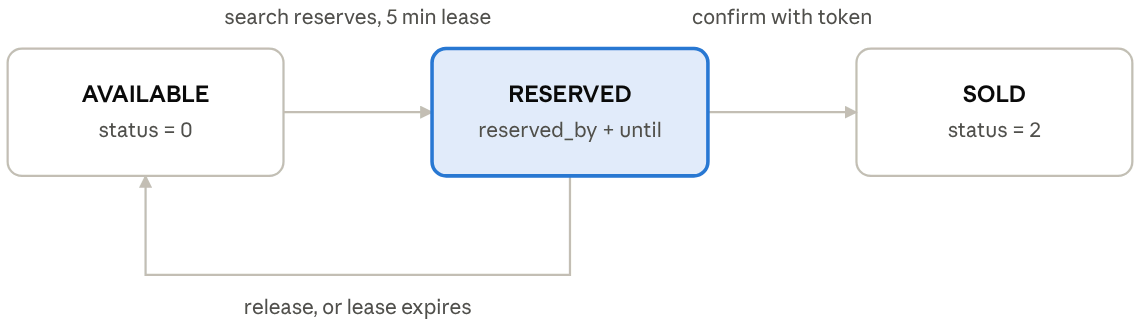
The bitmap approach is a strict improvement because it reduces every pattern, regardless of how many wildcards it has, to a constant-time in-memory operation followed by a small, targeted index lookup.

Consistency of the in-memory index. `AVAIL` may be up to one second stale on any given replica. That is safe by construction: it is only a filter for choosing candidates, and the database predicate `status = 0` is re-checked under a row lock at allocation time. A stale bit can cause the allocator to try a sold-out number and move on; it can never hand out a ticket that is not actually available.

6. Concurrency: reservation-based allocation

Duplicate simultaneous results are prevented by making the search itself an atomic reservation: a ticket returned to a user is locked for that user, in the database, before the response is sent. The guarantee is "no two active leases on one ticket", enforced by row locks during the transaction and by the lease columns after it commits.

A ticket is held by at most one lease at a time; the lease is the lock



ticket lifecycle · 3 states, 4 transitions

Only `RESERVED` can be entered concurrently, and the row lock plus `SKIP LOCKED` serialise that entry; every other transition is a compare-and-set on the token and the expiry.

The allocation statement. The allocator runs one transaction per search. In outline:

```
UPDATE tickets t
   SET status = 1,
       reserved_by = $user, reservation_id = $token,
       reserved_until = now() + interval '5 minutes',
       version = version + 1
  FROM (
    SELECT ticket_id FROM tickets
     WHERE draw_id = $draw
       AND number = ANY($candidate_numbers)
       AND (status = 0 OR (status = 1 AND reserved_until < now()))
     ORDER BY number, ticket_id
     LIMIT $batch_size
     FOR UPDATE SKIP LOCKED
  ) pick
 WHERE t.ticket_id = pick.ticket_id
RETURNING t.ticket_id, t.number;
```

Why this is correct:

- `FOR UPDATE` takes a row lock on each selected ticket. Two transactions cannot both select the same row.
- `SKIP LOCKED` means a transaction never waits on a row another one holds; it moves to the next matching row. Concurrent searches for the same pattern therefore get disjoint sets, in parallel, with no deadlocks and no retries.
- After commit, the `status = 1` and `reserved_until` columns act as the lock. Other searches exclude the row through the `WHERE` predicate until the lease expires.
- Expired leases are reclaimable in the same statement (`status = 1 AND reserved_until < now()`), so a crashed client never leaks tickets even if the sweeper is behind. Time comes from the database clock only, so there is no clock-skew problem across service replicas.

Confirm and release. `confirm(token)` is `UPDATE tickets SET status = 2 WHERE reservation_id = $token AND reserved_by = $user AND reserved_until > now()`. If zero rows update, the lease expired and the client is told to search again. `release(token)` clears the lease columns under the same guard. Both are idempotent, so retries are safe.

Hot-pattern contention. The weakness of a naive `ORDER BY` is that every user searching `****23` scans the same leading rows and skips the same locks, so throughput degrades as popular patterns fill with leases. Two mitigations, both cheap:

1. *Random start.* The matcher already picks a random starting number inside the result bitmap (Section 5). Concurrent requests land on different parts of the candidate space and rarely touch the same rows.
2. *Availability-filtered candidates.* Because `AVAIL` excludes sold-out numbers, the database is only asked about numbers with stock. The lock scan stays short even late in a draw.

If a product later requires strict fairness (first come, first served) instead of random spread, an `ORDER BY ticket_id` plus a modest batch size still works correctly; it only costs throughput on the hottest patterns.

Lease expiry sweeper. A background job runs every 30 s: `UPDATE tickets SET status = 0, reserved_by = NULL ... WHERE status = 1 AND reserved_until < now()`, using the partial index on `reserved_until`. It is a tidy-up, not a correctness mechanism, since the allocation predicate already treats expired leases as free.

Why not Redis locks? A `SET NX EX` per ticket in Redis is fast and atomic, but then ticket state lives in two places and a crash between the Redis write and the PostgreSQL write leaves them disagreeing. Keeping the lease inside the same transaction as the ticket row avoids distributed-lock edge cases entirely. Redis remains a reasonable option if a single PostgreSQL primary ever becomes the bottleneck, in which case the Lua script would reserve ticket IDs and PostgreSQL would be updated asynchronously through an outbox.

7. Performance analysis

A full search-and-reserve round trip costs a few milliseconds, dominated by one short PostgreSQL transaction; pattern matching itself is negligible. A single primary comfortably serves the 500 req/s target, and the service tier scales horizontally because every replica holds the same 7.5 MB index.

Step	Where	Cost per request	Notes
Pattern parse and validation	service	< 1 µs	6 characters

Step	Where	Cost per request	Notes
Bitmap AND (up to 7 bitmaps)	service, in memory	10–100 µs	15,625 words per bitmap; independent of ticket count
Random start + candidate window	service	< 50 µs	Walk set bits from a random offset
Reserve batch (`SKIP LOCKED`)	PostgreSQL	2–8 ms	Partial index lookup on ~100 numbers, 20 row locks, one commit
Confirm or release	PostgreSQL	< 2 ms	Single-token update
`AVAIL` refresh	loader → service	50–200 ms per second, off the request path	One grouped count query against the partial index

Throughput. Each reservation transaction locks about 20 rows and writes about 20 tuples. A mid-sized PostgreSQL primary (8 vCPU, data in cache) handles several thousand such transactions per second, so the 500 req/s peak leaves headroom of roughly 5–10×. Read-only endpoints (listing a user's own reservations, draw statistics) go to replicas.

Memory and storage. Per service replica: 61 bitmaps × 125 KB ≈ 7.5 MB, rebuilt at startup in under a second from one aggregate query. Database: about 1 GB of table data and 0.5 GB of indexes for 10M tickets, all cache-resident.

Scaling behaviour. The in-memory step is O(1) with respect to the number of tickets because it operates on the fixed 1M-number space. Going from 10M to 100M tickets changes nothing in the matcher and only grows the database by 10×, which is still a single-node workload. The partial index on available tickets shrinks as a draw sells out, so allocation gets cheaper over time rather than slower.

Worst cases and how they behave

- `******` (all wildcards): the bitmap result is `AVAIL` itself. Random start picks any number with stock; the database query is identical in cost to any other pattern.
- A nearly sold-out popular suffix: `AVAIL` filters most numbers out before the database is asked, so the lock scan touches only numbers that still have stock. When the result bitmap is empty the service returns "no tickets left" without a database round trip.

- A burst of identical patterns: random start spreads lock traffic across the candidate space; `SKIP LOCKED` guarantees no waiting. Contention shows up as slightly smaller batches, never as blocked requests.
- Stale `AVAIL` after a sale: the allocator may ask for one or two numbers that just sold out and get fewer rows back. The service pads the batch from the next window; this is a second cheap query, not a failure.

Trade-offs accepted

- A replica-local index means a replica can be up to one second behind. Accepted, because correctness is enforced in the database; staleness only affects candidate selection.
- Random assignment instead of strict FIFO. Accepted for throughput; FIFO is a one-line change if the product requires it.
- One primary for all writes. Accepted at this scale; the escape hatch is partitioning by draw and, beyond that, moving reservations to Redis with an outbox.

8. Alternatives considered and operational notes

The simplest design that meets every requirement is the one above; the alternatives below were rejected for a specific reason each, and most remain available as later upgrades.

Alternative	Why not as the primary design	When I would revisit it
PostgreSQL only, per-digit indexes	100× more index work per query; planner-dependent performance on wide patterns	Never for search; fine as a cold fallback if a replica loses its index
Trie / prefix tree over numbers	Fast for prefix patterns, poor for suffix or mixed patterns without six rotated tries	Only if patterns were prefix-only
Elasticsearch for search + database for allocation	Two systems to keep consistent; refresh lag means users see tickets that are already gone	If free-text or fuzzy search across other ticket fields became a requirement

Alternative	Why not as the primary design	When I would revisit it
Redis as the reservation authority	Distributed state between Redis and PostgreSQL; durability trade-offs for the record of sales	If one PostgreSQL primary could no longer absorb the write rate
Pre-sharded queues per pattern	Combinatorial number of patterns (over 1M shapes); most would be empty	Not practical

Operating it in production

- *Deployment.* The service is stateless and runs as a Kubernetes Deployment with 3+ replicas behind the gateway; readiness only turns green once the bitmaps are built. PostgreSQL runs as a managed primary with one synchronous or near-synchronous replica for failover.
- *Draw rollover.* Tickets for a new draw are bulk-loaded into a fresh partition while the current draw is still selling; a feature flag switches the active `draw_id`. Old partitions are detached and archived.
- *Observability.* The metrics that matter: search p99 latency, reservation batch fill rate (rows returned ÷ rows requested), lock-skip rate, lease expiry rate, and `AVAIL` refresh age. A rising expiry rate usually means the lease is too long for the checkout flow; a falling fill rate on hot patterns means the candidate window should grow.
- *Abuse.* Rate-limit searches per user at the gateway and cap outstanding reservations per user, otherwise a single client could hold a popular suffix hostage for the lease duration.
- *Failure modes.* A service replica crash loses nothing: leases live in the database and expire on their own. A database failover keeps committed leases; in-flight transactions roll back and clients simply retry the search.